M321 · Microservice Patterns anwenden

5IA23f · 21. Oktober 2026 · Projekt: Service 01 / Service 02 / Eureka / Gateway / React

Drei implementierte Patterns mit Problem, tatsächlichem Code, automatisierten Tests und Reflexion. Die abgebildeten Ausschnitte stammen aus dem Repository und den unten genannten Testläufen.

Aufgabenquelle: https://bbw-it.github.io/321_main_rupe/14_Pattern/14.4_MicroservicesPatternsAnwenden/

1. Logging / Observability — Service 01

Problem: Ohne Request-Logs sind Fehler, langsame Antworten und Aufrufwege schwer nachvollziehbar.

Implementierung: Ein Servlet-Filter protokolliert Requests mit Methode, Pfad, Status und Laufzeit; Bodies und Zugangsdaten werden nicht geloggt.

Implementierung_1

```
$ RequestLoggingFilter.java
 23          try {
 24              chain.doFilter(request, response);
 25          } finally {
 26              long durationMs = TimeUnit.NANOSECONDS.toMillis(System.nanoTime(…
 27              log.info("{} {} status={} durationMs={}", request.getMethod(), r…
 28                      response.getStatus(), durationMs);
 29          }
 30      }
 31  }
```

Abbildung: Implementierungs-Ausschnitt aus service01/src/main/java/ch/bbw/pr/eureka/service01/RequestLoggingFilter.java

Test: POST /books liefert 201; das Log enthält POST, Pfad, Status und Dauer, aber weder Query-Token noch Buchinhalt.

Testcode_1

```
$ RequestLoggingTest.java
 28                          .content("{\"id\":99,\"title\":\"" + secret + "\",\"…
 29                  .andExpect(status().isCreated());
 30
 31          assertThat(output.getOut()).containsPattern("POST /books status=201 …
 32          assertThat(output.getOut()).doesNotContain(secret, "private-query-sh…
 33      }
 34  }
```

Abbildung: Test-Ausschnitt aus service01/src/test/java/ch/bbw/pr/eureka/service01/RequestLoggingTest.java

Testlauf_1

```
$ ./mvnw verify
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Abbildung: Ausgabe des echten Maven-Testlaufs (gekürzt).

Reflexion: Logging macht eine Ursache sichtbar und unterstützt Diagnose, verhindert aber keinen Ausfall. Die gemessene Dauer hilft bei der Suche nach langsamen Endpunkten.

Nachweis: service01/src/main/java/ch/bbw/pr/eureka/service01/RequestLoggingFilter.java · service01/src/test/java/ch/bbw/pr/eureka/service01/RequestLoggingTest.java

2. Timeout — Service 02 zu Service 01

Problem: Bleibt der Buchdienst hängen, darf Service 02 nicht unbegrenzt auf die Feign-Antwort warten.

Implementierung: Feign erhält einen Read-Timeout von 250 ms für service-01; normale Aufrufe bleiben unverändert.

Implementierung_2

```
$ application.properties
  3  eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
  4  eureka.instance.lease-renewal-interval-in-seconds=5
  5  # Limit only the service-01 Feign client's wait for response bytes (millisec…
  6  spring.cloud.openfeign.client.config.service-01.readTimeout=250
```

Abbildung: Implementierungs-Ausschnitt aus service02/src/main/resources/application.properties

Test: Eine simulierte Antwort nach 1 s löst bei Feign einen SocketTimeoutException aus; normale GET/POST-Aufrufe funktionieren weiter.

Testcode_2

```
$ BooksClientTimeoutTest.java
 45      }
 46
 47      @Test
 48      void slowService01ResponseTimesOut() throws InterruptedException {
 49          SLOW_RESPONSE.set(true);
 50          try {
 51              RetryableException timeout = assertThrows(RetryableException.cla…
 52              assertTrue(timeout.getCause() instanceof SocketTimeoutException,
 53                      () -> "Expected a socket read timeout, got " + timeout);
 54          } finally {
```

Abbildung: Test-Ausschnitt aus service02/src/test/java/ch/bbw/pr/eureka/service02/BooksClientTimeoutTest.java

Testlauf_2

```
$ ./mvnw verify
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Abbildung: Ausgabe des echten Maven-Testlaufs (gekürzt).

Reflexion: Der Timeout begrenzt die Wartezeit und schützt Aufrufer-Ressourcen. Er repariert den langsamen Downstream nicht; ohne Fallback erhält der Client weiterhin einen Fehler.

Nachweis: service02/src/main/resources/application.properties · service02/src/test/java/ch/bbw/pr/eureka/service02/BooksClientTimeoutTest.java

3. Rate Limiting — API-Gateway

Problem: Zu viele Anfragen können Backend-Dienste überlasten oder faire Nutzung verhindern.

Implementierung: Ein Gateway-Filter begrenzt Requests pro Client und gibt bei Überschreitung HTTP 429 zurück; die bestehenden Eureka-Routen bleiben erhalten.

Implementierung_3

```
$ ClientRateLimitFilter.java
 38          }
 39          InetSocketAddress remote = exchange.getRequest().getRemoteAddress();
 40          String client = remote == null ? "unknown" : remote.getHostString();
 41          if (allow(client)) {
 42              return chain.filter(exchange);
 43          }
 44          exchange.getResponse().setStatusCode(HttpStatus.TOO_MANY_REQUESTS);
 45          return exchange.getResponse().setComplete();
 46      }
 47
 48      private synchronized boolean allow(String client) {
 49          long window = Math.floorDiv(clock.millis(), WINDOW_MILLIS);
```

Abbildung: Implementierungs-Ausschnitt aus gateway/src/main/java/ch/bbw/pr/eureka/gateway/ClientRateLimitFilter.java

Test: Fünf Requests pro IP und Minute passieren; der sechste erhält HTTP 429. Andere IPs und das nächste Minutenfenster bleiben nutzbar.

Testcode_3

```
$ ClientRateLimitFilterTests.java
 32          for (int i = 0; i < 5; i++) {
 33              assertThat(request("127.0.0.1").getResponse().getStatusCode()).i…
 34          }
 35          assertThat(request("127.0.0.1").getResponse().getStatusCode()).isEqu…
 36          assertThat(request("127.0.0.2").getResponse().getStatusCode()).isNul…
 37          assertThat(forwarded).hasValue(6);
 38      }
 39
 40      @Test
 41      void quotaResetsAtNextMinuteWithoutSleeping() {
```

Abbildung: Test-Ausschnitt aus gateway/src/test/java/ch/bbw/pr/eureka/gateway/ClientRateLimitFilterTests.java

Testlauf_3

```
$ ./mvnw verify
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Abbildung: Ausgabe des echten Maven-Testlaufs (gekürzt).

Reflexion: Überzählige Requests werden aktiv abgewiesen, damit Backends entlastet werden. Ein lokaler In-Memory-Zähler ist für eine einzelne Gateway-Instanz geeignet, aber nicht global über mehrere Instanzen hinweg.

Nachweis: gateway/src/main/java/ch/bbw/pr/eureka/gateway/ClientRateLimitFilter.java · gateway/src/test/java/ch/bbw/pr/eureka/gateway/ClientRateLimitFilterTests.java